

实验7：文件系统

实验7：实现一个功能完整的文件系统

概述

项目	说明
实验编号	Lab 7 of 8
框架提供的	<code>kernel/fs/bio.c</code> （含 TODO 的 5 个函数）、 <code>kernel/fs/fs.c</code> （ <code>bmap</code> / <code>readi</code> / <code>dirlookup</code> 含 TODO 的骨架）
需要补全 TODO 的函数	<code>binit</code> 、 <code>bget</code> 、 <code>bread</code> 、 <code>bwrite</code> 、 <code>brelease</code> （块缓冲层，5 处 TODO）
需要自主设计的函数	<code>balloc</code> / <code>bfree</code> / <code>bzero</code> （块分配）、 <code>iget</code> / <code>ilock</code> / <code>iunlock</code> / <code>iput</code> / <code>iupdate</code> / <code>ialloc</code> / <code>fsinit</code> （inode 管理）、 <code>writei</code> / <code>bmap</code> 、 <code>dirlookup</code> （补 TODO）、 <code>skipelem</code> / <code>namei</code> / <code>nameiparent</code> / <code>dirlink</code> （目录与路径）、 <code>filealloc</code> / <code>fileclose</code> / <code>fileread</code> / <code>filewrite</code> （文件描述符层）、 <code>sys_open</code> / <code>sys_read</code> / <code>sys_write</code> / <code>sys_close</code> （系统调用）
新建文件	<code>kernel/fs/file.c</code> 、 <code>kernel/syscall/sysfile.c</code>
验收标准	用户进程能执行 <code>open</code> / <code>write</code> / <code>read</code> / <code>close</code> ，在虚拟磁盘中创建文件、写入数据；系统重启后重新读取，数据不丢失。

阅读警告：本实验的理论和实践之间存在大量“看似懂了但一写就错”的鸿沟。这些鸿沟会在每个章节中用 ⚠️ 标记出来，请务必认真阅读——它们是无数学生踩过之后总结出来的经验。

读本手册的方式

本手册不是“填空指南”——它是一本**导游册**，带你从“用户发出 `write()` 申请”一路走到“字节落到磁盘某个扇区”。

整个旅程分为 6 个站点（Layer 0~5）。每到一站，我们先问：“这一层解决了什么问题？”，再问：“它提供了哪些接口，内部又是怎么工作的？”，最后才问：“你需要写哪些代码？”对于有框架 TODO 的部分，你按指引补全即可。对于没有框架 TODO 的部分，手册会指引你去阅读 xv6 的对应源文件，理解设计逻辑后自己建模实现——不需要照搬，但需要理解“为什么这样设计”。

xv6 源码参考路径 xv6-riscv 仓库（<https://github.com/mit-pdos/xv6-riscv>）中，文件系统核心在：

- - `kernel/bio.c` — 块缓冲层
- - `kernel/fs.h` — 磁盘数据结构定义
- - `kernel/fs.c` — inode 管理、块分配、读写、目录、路径解析
- - `kernel/file.h` — 文件描述符数据结构
- - `kernel/file.c` — 文件描述符层
- - `kernel/sysfile.c` — 文件相关系统调用

⚠ 理论和实践的鸿沟在哪里？

这是本手册最重要的一段话，请先读一遍，再开始动手。

文件系统是操作系统所有实验中对“直觉”挑战最大的部分。原因是：教材讲的和真正做的之间有三层转化，每一层都可能出错。

以下是本实验涉及的 9 道理论和实践之间的鸿沟，每一道都是往届学生踩过的坑：

编号	鸿沟	常见错误
1	”文件系统”是一条调用链，不是一个模块	只实现了某一层，忽略了其他层必须存在
2	inode 的 <code>addrs[]</code> 存的是 磁盘块号 ，不是内存指针	调用 <code>bread(ip->dev, ip->addrs[3])</code> 时当成指针用
3	缓冲层保证同一磁盘块在内存中只有一份 拷贝	以为缓冲层只是性能优化，忽略其对一致性的关键作用
4	目录是文件，但内核对目录的访问有 特殊限制	尝试用 <code>writei</code> 直接写目录文件
5	写文件是 整块读写 ，不是字节追加	以为 <code>write</code> 可以随意写任意 offset，每次写入都以为会自动追加到末尾
6	<code>iget</code> 不读磁盘 ， <code>ilock</code> 才读磁盘	在 <code>iget</code> 中直接调用 <code>bread</code> 读磁盘，导致重复读取和缓存不一致
7	目录项名字比较用 固定字节数 （DIRSIZ），不是 <code>strcmp</code>	用 <code>strcmp</code> 比 <code>de.name</code> ，导致误匹配或漏匹配
8	<code>skipelem</code> 有三个边界：正常路径、尾部斜杠、空字符串	只处理了第一种情况，实现不完整
9	<code>fd</code> 是 进程打开文件的实例 ，不是 inode 编号的别名	省去 <code>struct file</code> 层，导致多进程无法独立维护偏移

本手册的结构：每个章节都会先用一段”原理”建立直觉，然后用 ⚠️ 标记出这一部分最常见的理解偏差，最后给出具体的实现细节。这样你就能知道：

- 教科书上的理论在哪里简化了
- 框架代码在哪里反映了真实复杂性
- 你填进去的代码在哪里可能出错

一次 `write()` 调用的完整旅程

在开始任何编码之前，先用这张地图在脑中走一遍全程：

```
1  用户进程调用  write(fd=3, buf="hello", n=5)
2      ↓ ecalls → S-Mode
3      [Layer 5]  系统调用层 sys_write()
```

```

4      ↓ 从进程文件描述符表取出 struct file
5      [Layer 4] 文件描述符层 fwrite(f, buf, 5)
6      ↓ 找到对应 inode, 调用 writei
7      [Layer 3] inode层 writei(ip, buf, offset, 5)
8      ↓ 计算逻辑块号, 调用 bmap 得到物理块号
9      [Layer 3] 块映射 bmap(ip, bn) → 物理块号 blockno
10     ↓ 若块未分配, 先调用 balloc
11     [Layer 2] 块分配 balloc(dev) → 分配一个空闲磁盘块
12     ↓ 通过缓冲层读出该块
13     [Layer 1] 块缓冲 bread(dev, blockno) → struct buf*
14     ↓ 修改 buf->data 中的字节
15     ↓ 持久化
16     [Layer 1] 块缓冲 bwrite(b)
17     ↓ 写入虚拟磁盘驱动
18     [Layer 0] virtio_disk_rw(b, write=1) ← 框架已提供, 无需实现

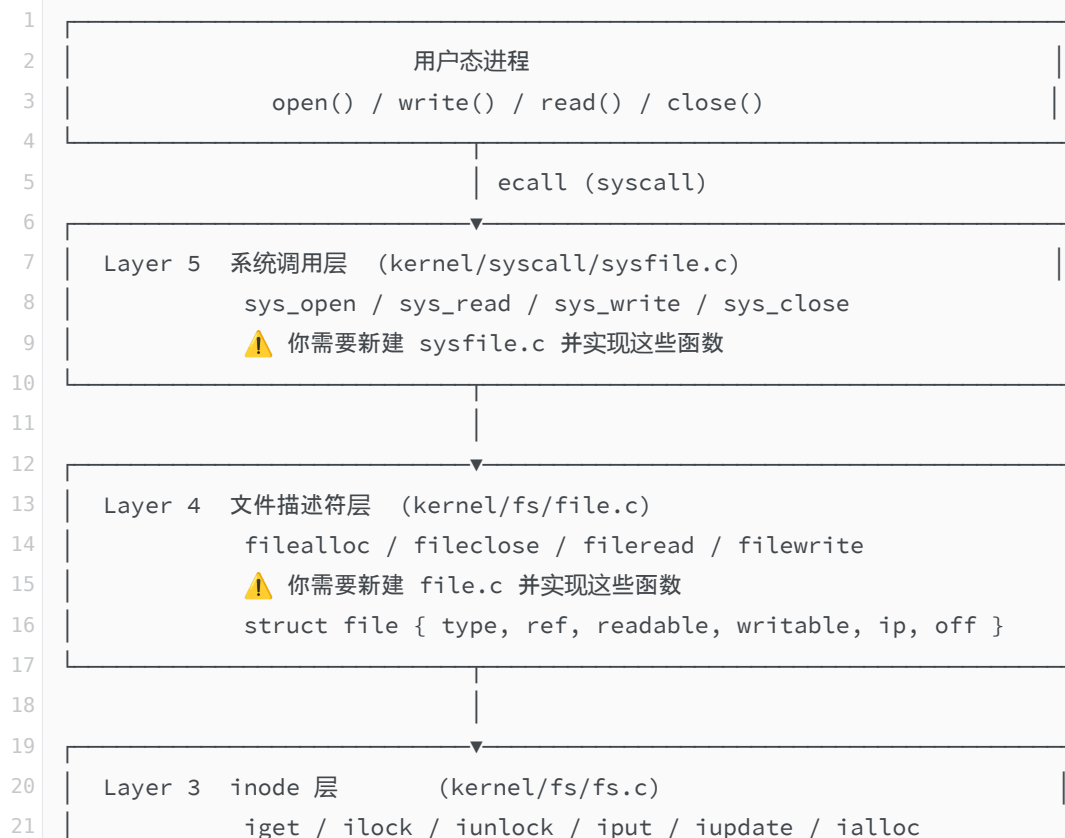
```

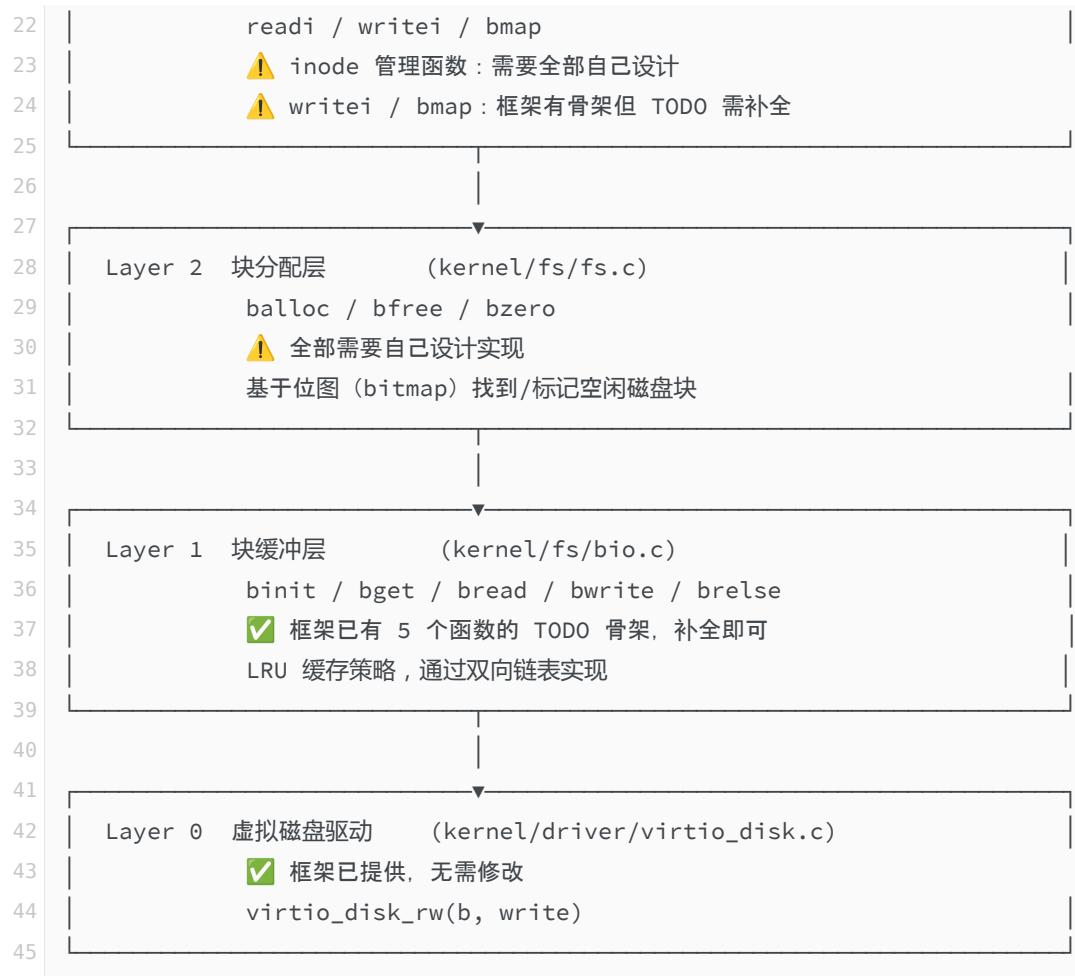
每一层都是一个**抽象屏障**：上层不需要知道下层的细节，只需要调用下层的接口。这样，当你实现 `writei` 时，只需关注“块在哪里”，不需要关心磁盘驱动怎么工作。

第一节：框架全貌——哪些已提供，哪些要自己写

在动手之前，你必须对整个文件系统的代码结构有一个清晰的地图。本节给出完整的层次图和代码位置对照。

1.1 六层架构总览





1.2 代码位置总表

函数	文件	状态
<code>binit</code> / <code>bget</code> / <code>bread</code> / <code>bwrite</code> / <code>brelease</code>	<code>kernel/fs/bio.c</code>	框架有 TODO，补全
<code>fsinit</code> / <code>ballocc</code> / <code>bfree</code> / <code>bzero</code>	<code>kernel/fs/fs.c</code>	全部自己设计
<code>iget</code> / <code>ilock</code> / <code>iunlock</code> / <code>iput</code> / <code>iupdate</code> / <code>iallocc</code>	<code>kernel/fs/fs.c</code>	全部自己设计
<code>bmap</code>	<code>kernel/fs/fs.c</code>	框架有 TODO 骨架，补全
<code>readi</code>	<code>kernel/fs/fs.c</code>	框架有 TODO 骨架，补全
<code>writel</code>	<code>kernel/fs/fs.c</code>	全部自己设计
<code>dirlookup</code>	<code>kernel/fs/fs.c</code>	框架有 TODO 骨架，补全
<code>dirlink</code> (目录中注册新条目)	<code>kernel/fs/fs.c</code>	必做 —— <code>sys_open(0_CREAT)</code> 创建文件所必需
<code>skipelem</code> / <code>namei</code> / <code>nameiparent</code>	<code>kernel/fs/fs.c</code>	全部自己设计
<code>fileallocc</code>	<code>kernel/fs/file.c</code>	全部自己设计（新建文件）

/ fileclose / fileread / filewrite		
sys_open / sys_read / sys_write / sys_close	kernel/syscall/sysfile. c	全部自己设计（新建文件）

1.3 ⚠ 理论和实践的第一道鸿沟：“文件系统”不是一个东西

很多学生在学完操作系统原理后，以为“文件系统”是一个单独的模块。实际上，文件系统是一整条调用链——从用户态的 `open` 到磁盘写入，中间经过了 5 个层次，每层都有自己的状态管理。

这条调用链中，没有任何一层可以省略。省略了文件描述符层，你就无法支持两个进程各自独立偏移；省略了 inode 层，你就无法找到文件的数据块在哪；省略了块缓冲层，每次读写都要真正操作磁盘（极慢）。

当你实现某一层时，必须同时维护好：

- 本层状态（如 `struct file` 中的 `off` 偏移）
- 对下层的调用约定（如调用 `ilock` 前必须先 `iget`）

违反约定不会报编译错误，但运行时会出现难以调试的 data corruption 或 panic。

第二节：磁盘布局与关键数据结构——从物理存储到逻辑分区

xv6 风格的文件系统将磁盘划分为固定的逻辑区域，各区域位置在格式化时（`mkfs`）确定，之后不会改变。

2.1 磁盘的基本单位：块（Block）

磁盘不像内存那样可以按字节随机访问，它以固定大小的块为最小读写单位。在本框架（和 xv6）中，一个块 = 1024 字节（`#define BSIZE 1024`）。

所有磁盘操作都是以块为单位的：你不能“只修改一个字节”——你必须先把整个块读入内存，修改后整块写回。

2.2 磁盘的逻辑分区

用 `mkfs` 格式化磁盘时，会把整个磁盘划分成如下区域（位置在格式化时确定，之后不会变动）：

1	磁盘块编号：0	1	2~31	32~44	45	46~999
2						

3		Boot	Super	日志区	Inode区	位图	数据区
4		Block	Block	(Log)		块	(Data)
5							

每个区域的职责：

区域	职责
Boot Block (块0)	引导加载程序 (BIOS 固定从这里读)
Super Block (块1)	文件系统的“说明书”：总块数、inode 数、各区域起始块号
日志区	崩溃恢复 (本实验可简化略过)
Inode 区	存储所有文件的“元数据” (大小、数据块位置等)
位图区 (Bitmap)	每个 bit 对应一个数据块：1=已用，0=空闲
数据区	文件的实际内容，以及目录的条目列表

超级块 (Superblock) 是最重要的元数据，存储整个文件系统的元数据，文件系统初始化时必须先读取它：

```

1 // 来自 xv6 kernel/fs.h — 磁盘超级块结构
2 struct superblock {
3     uint magic;           // 魔数，文件系统类型验证码 (必须 == FSMAGIC, 验证这确实是
4     uint size;            // 磁盘总块数
5     uint nblocks;         // 数据块总数
6     uint ninodes;         // 最大 inode 数
7     uint nlog;            // 日志区块数
8     uint logstart;        // 日志区起始块号
9     uint inodestart;      // inode 区起始块号
10    uint bmapstart;       // 位图区起始块号
11 };

```

初始化时，内核将这个结构读入内存（存在全局变量 `struct superblock sb` 中），此后所有“找第 N 个 inode 在哪里”的计算都依赖它。

`struct superblock` 的定义已在上方给出。你需要在 `kernel/fs/fs.c` 中声明全局变量 `struct superblock sb;` 并在 `fsinit` 中从磁盘加载。

思考：为什么超级块固定在块1而不是块0？如果块1的数据损坏了，会发生什么？

2.3 ⚠ 理论和实践的第二道鸿沟：inode 编号不是内存指针

教科书上画 inode 通常是一个方块，里面写着“地址列表”。这个图容易造成一个误解：inode 里的 `addrs[]` 存的是内存地址。

实际上, `addr[]` 存的是磁盘块号 (整数)。这些块号指向磁盘上的数据块, 不是内存指针。

举例来说, `ip->addr[3] == 47` 的意思是: "这个文件的第 3 个逻辑块, 存储在磁盘的第 47 块。"

在代码中, 当你调用 `bread(ip->dev, ip->addr[3])` 时, 第二个参数是块号 47, 不是某个指针。

另外, `addr[]` 还有直接块和间接块的区别 (见下节), 不是简单的一张表。

2.4 从分区到函数的映射

现在你已经知道磁盘的物理布局, 下面将每个区域与你需要实现的函数对应起来:

磁盘区域	操作	你需要实现的函数
任意块	缓存读/写	bread 、 bwrite 、 brelse (块缓冲层, 框架 TODO)
数据区	分配/释放空闲块	balloc 、 bfree 、 bzero (自主设计)
Inode 区	读取/修改 inode 元数据	iget 、 ilock 、 iput 、 iupdate (自主设计)
数据区	通过 inode 读写文件内容	readi (框架 TODO) 、 writei (自主设计)
数据区	在目录中查找文件名	dirlookup (框架 TODO)
全路径	解析路径字符串 /usr/README	namei (自主设计)
内存	文件描述符管理	filealloc 、 fileread 、 filewrite (自主设计)

第三节：块缓冲层——框架 TODO（先完成它）

3.1 为什么需要缓冲层？

磁盘访问非常慢（相对 CPU 来说），直接每次都调用 `virtio_disk_rw()` 代价很高。解决方案是在内存中维护一个**缓冲池（Buffer Cache）**：

- 第一次读块 X → 从磁盘读，存入内存缓冲
- 第二次读块 X → 直接从内存返回，不再读磁盘
- 修改块 X 的内容 → 在内存中修改，最后批量写回磁盘

缓冲层还有一个关键作用：**保证同一时刻每个磁盘块在内存中只有一份拷贝**。否则两个进程分别修改同一块的两份拷贝，写回时就会产生冲突。

3.2 ⚠ 理论和实践的第三道鸿沟：缓冲层不是“缓存”而是“唯一性保证”

很多同学以为缓冲层就是一个性能优化——让磁盘读写变快一点。这个理解是**不完整的**。

缓冲层更核心的作用是“同一磁盘块在内存中永远只有一份拷贝”。

这意味着：当两个进程同时读同一个磁盘块时，它们最终拿到的是同一个 `struct buf` 指针。

当一个进程写了这个块，另一个进程下次读取会看到修改。

如果没有这个机制，两个进程可能同时修改同一个磁盘块的两份拷贝，然后在 `brelse` 时分别写回磁盘——结果就是 data corruption。缓冲层把这个“二义性”消除了。

3.3 缓冲池的数据结构

```
1 // bio.c 中已定义
2 struct {
3     struct buf buf[NBUF]; // NBUF 个缓冲块（固定数量，如30个）
4     struct buf head;      // 双向链表哨兵头节点
5 } bcache;
```

每个 `struct buf` 代表内存中缓存的一个磁盘块：

```
1 struct buf {
2     int    valid;          // 1=内存中的数据与磁盘同步，0=需要重新从磁盘读
3     uint   dev;            // 设备号
4     uint   blockno;        // 磁盘块号
5     uint   refcnt;         // 引用计数（有多少人正在使用）
6     struct buf *prev;      // LRU 链表前驱
7     struct buf *next;      // LRU 链表后继
8     uchar  data[BSIZE];    // 磁盘块数据（1024字节）
9 };
```

链表的排列用于实现 LRU（最近最少使用）淘汰策略：

```
1 head ↔ [最近刚用完 A] ↔ [较新 B] ↔ [较旧 C] ↔ [最久没用 D] ↔ head
2     ↑ brelse 后头插                                ↑ bget 从尾部扫描淘汰
```

规则很简单：**刚用完的块插到链表头部**（它最近被访问，暂时不淘汰）；**需要淘汰时从链表尾部找 `refcnt==0` 的块**（最久没人用的那个）。

2.3 完成框架 TODO（`bio.c`）

参考 xv6：打开 `xv6-riscv/kernel/bio.c`，完整阅读 `binit`、`bget`、`bread`、`bwrite`、`brelse` 五个函数。本框架的结构与 xv6 几乎相同。

`binit()` (TODO 步骤1)：把 `bcache.buf[]` 中的每个 buf 插入到哨兵头后面。头插法每次修改 4 个指针：

- 1 插入前：head ↔ 已有节点
- 2 插入后：head ↔ 新节点 ↔ 已有节点
- 3 需要修改：新节点.prev，新节点.next，head.next，原来第一个节点.prev

`bget()` (TODO 步骤2)：找到 `refcnt==0` 的空闲块重用时，需要：

1. 将它的 `dev`、`blockno` 更新为新目标
2. 将 `valid` 设为 0 (旧数据已失效，必须重新从磁盘读)
3. 将 `refcnt` 设为 1

`bread()` (TODO 步骤3)：若 `b->valid==0`，调用 `virtio_disk_rw(b, 0)` 读磁盘，再将 `valid` 置为 1。

`bwrite()` (TODO 步骤4)：调用 `virtio_disk_rw(b, 1)` 写磁盘。注意：只写，不改 `valid` (写后数据仍然有效)。

`brelease()` (TODO 步骤5)：`refcnt--`；若归零，将该 buf 从当前链表位置摘除，重新头插到 `bcache.head` 后面 (这一步就是 LRU 的核心)。

验证方法：完成 `binit` 和 `bread` / `brelease` 后，在 `fsinit` (下一节你会实现) 中读取超级块 (块1)，检查 GDB 中 `sb.magic` 值是否为 `0x10203040` (xv6 文件系统魔数)。

第四节：inode 和位图——理解文件系统的核心抽象

在实现任何“有意义”的文件系统功能之前，必须先理解 **inode (索引节点)** 是什么。

4.1 用户认知的“文件”与内核认知的“文件”

用户认为文件 = 一个名字 + 一段内容。但这个朴素认知忽略了两件事：

1. 同一个文件可能有多个名字 (硬链接)
2. 内容可能散落在磁盘的不同块上

内核的认知：文件 = 一个 **inode (元数据)** + 若干数据块。

- **inode 存储**：文件大小、类型 (普通文件/目录/设备)、数据块的位置列表
- **名字** 只是目录文件中一条记录，指向某个 inode 编号 (inum)
- **inode 与名字之间没有绑定关系**——一个 inode 可以对应多个名字 (硬链接)，也可以没有任何名字 (孤立文件，等待 iput 回收)

4.2 ⚠ 理论和实践的第四道鸿沟：目录是“文件”这句话的真实含义

“目录也是文件”是教科书上常见的一句话。但它的真实含义远比字面意思更微妙：

字面意思：目录的 inode 类型是 `T_DIR`，它的数据块里存着 `struct dirent` 数组。

更深的含义：正是因为目录是文件，所以你可以用 `readi` 来读目录——这也是 `dirlookup` 的实现原理。但你不能用 `writei` 来“写”目录 (xv6 禁止向目录文件写入)，因为目录的结构必须严格遵循 `struct dirent` 的格式，随意写入会破坏文件系统的内部一致性。

所以更准确的表述是：目录在磁盘上的物理存在形式与普通文件完全一样 (都是 inode + 数据块)，但内核对它的访问方式做了限制 (只读、只用 `dirent` 格式解析)。

4.3 磁盘上的 inode (`struct dinode`)

磁盘 inode 区中紧密排列着每个文件的元数据：

```

1 // 来自框架 fs.c (与 xv6 kernel/fs.h 的 struct dinode 一致)
2 struct dinode {
3     short type;           // 文件类型: 0=空闲, 1=普通文件, 2=目录, 3=设备
4     short major, minor;   // 设备号 (type==3 时有效)
5     short nlink;          // 硬链接计数
6     uint size;            // 文件大小 (字节)
7     uint addrs[NDIRECT+1]; // 数据块地址表
8 };

```

`addrs[]` 数组存储文件数据的位置:

```

1 addrs[0]          → 数据块 #0 的磁盘块号 (直接指针)
2 addrs[1]          → 数据块 #1 的磁盘块号
3 ...
4 addrs[11]         → 数据块 #11 的磁盘块号 (最后一个直接指针)
5 addrs[12]         → 一级间接块的磁盘块号 (间接指针)
6                   这个间接块里面存着 256 个数据块号
7
8 最大文件大小 = (12 + 256) × 1024 = 268 KB

```

inode 编号与磁盘位置的对应关系:

```

1 // inode #inum 所在的磁盘块号 (IPB = BSIZE / sizeof(struct dinode))
2 uint blockno = sb.inodestart + inum / IPB;
3 // 该 inode 在这个块中的偏移位置
4 struct dinode *dip = (struct dinode*)bp->data + inum % IPB;

```

这两个公式会在你实现 `ilock` 和 `iupdate` 时频繁用到。

3.3 内存中的 inode (`struct inode`)

磁盘 inode 不能直接当作运行时数据结构使用——磁盘读写太慢, 而且需要引用计数等运行时信息。因此内核维护一个 **inode 缓存 (icache)**, 每个 `struct inode` 是对应磁盘 inode 的内存副本:

```

1 // 来自框架 fs.c
2 struct inode {
3     uint dev;   // 设备号
4     uint inum;  // inode 编号 (对应磁盘位置的"身份证")
5     int ref;    // 引用计数: 有多少进程持有指向这个 inode 的指针
6     int valid;  // 磁盘内容是否已加载到这个结构体
7     // 以下字段从磁盘复制 (ilock 时加载)
8     short type;
9     short major, minor, nlink;
10    uint size;
11    uint addrs[NDIRECT+1];
12 };

```

`ref` 和 `valid` 的区别：

- `ref > 0`：有人持有这个内存 inode 的指针（调用 `iget` 后 `ref+1`，调用 `iput` 后 `ref-1`）
 - `valid == 1`：磁盘内容已经加载到这个结构体里（调用 `ilock` 后才保证）
- 一个 inode 可能是：`ref==1, valid==0`（刚从 `iget` 获取，还没加载磁盘数据），或 `ref==1, valid==1`（已加载，可以使用），或 `ref==0`（缓存槽空闲，可以被 `iget` 占用）。
-

第五节：磁盘块分配——自主设计（`balloc` / `bfree` / `bzero`）

5.1 问题：文件占用的磁盘块从哪里来？

当 `writei` 需要给文件追加内容时，可能需要分配新的磁盘块。位图区记录了哪些数据块空闲（`bit=0`）哪些已用（`bit=1`）。

`balloc(dev)` 的任务：找到第一个 `bit==0` 的位置，将其标记为1，并返回对应的块号。

4.2 位图的工作原理

位图中每个字节控制 8 个数据块的状态，每个位图块（1024字节）控制 `BPB = 1024 × 8 = 8192` 个数据块。

对于块号 `b`：

- 它在哪个位图块：`sb.bmapstart + b / BPB`
- 在那个位图块的哪个字节：`(b % BPB) / 8`
- 在那个字节的哪一位：`b % 8`

找到空闲块的算法（伪代码）：

```
1 for 每个位图块 bi :
2     bp = bread(dev, 位图块号)
3     for 字节 m in bp->data[0..BSIZE-1] :
4         for 位 bi2 in 0..7 :
5             如果 m 的第 bi2 位 == 0: // 找到空闲块
6                 将该位置1
7                 bwrite(bp)           // 写回位图
8                 brelse(bp)
9                 bzero(dev, 对应块号) // 清零新分配的块
10                return 对应块号
11 panic("balloc: out of blocks")
```

参考 xv6：阅读 `xv6-riscv/kernel/fs.c` 中的 `balloc` 函数（约30行），理解位运算：
`if((b->data[bi/8] & (1 << (bi%8))) == 0)` — 检查某位是否为0
`>data[bi/8] |= (1 << (bi%8))` — 将某位置1

`bzero(dev, bno)`：分配新块后立即清零，防止“前一个文件的旧数据泄露给新文件”。实现：

`bread` → `memset(bp->data, 0, BSIZE)` → `bwrite` → `brelse`。

`bfree(dev, b)`：与 `balloc` 相反，将位图中对应位清零。在实现文件删除（`unlink`）时需要。

5.3 把这三个函数放在哪里？

在 `kernel/fs/fs.c` 中，在 `bmap` 函数之前定义 `balloc`、`bfree`、`bzero`（均为 `static`，因为这是文件系统内部函数）。

第六节：inode 生命周期管理——自主设计

这是整个文件系统最核心也最精妙的部分。建议先把 xv6 的实现完整读懂再动手。

参考 xv6：重点阅读 `xv6-riscv/kernel/fs.c` 中的以下函数（按阅读顺序）：

1. `iinit`（inode 缓存初始化，对应你的 `fsinit` 中的一部分）
2. `iget`（获取 inode 缓存项）
3. `ilock`（锁定并从磁盘加载 inode）
4. `iunlock`（解锁）
5. `iput`（释放引用）
6. `iupdate`（将内存修改写回磁盘）
7. `ialloc`（在磁盘上分配新 inode）

6.1 你需要维护的全局数据结构

在 `fs.c` 中添加 inode 缓存：

```
1 // 你需要在 fs.c 中声明（类似 xv6 kernel/fs.c 的 icache）
2 struct {
3     struct inode inode[NINODE]; // NINODE 在 param.h 中定义
4 } icache;
5
6 // 全局超级块（在 fsinit 时从磁盘读入）
7 struct superblock sb;
```

6.2 `fsinit(int dev)` — 文件系统的起点

这是系统启动时第一个被调用的文件系统函数，让“inode 缓存”和“超级块信息”就位。

职责清单：

1. 调用 `binit()` (初始化块缓冲池)
 2. 读取超级块: `bread(dev, 1)` 读取磁盘块1, 用 `memmove` 将 `bp->data` 复制到全局 `sb`, 然后 `brelse(bp)`
 3. 验证魔数: `if(sb.magic != FSMAGIC) panic("invalid file system")`
 4. 将 `icache.inode[]` 全部清零 (`ref=0, valid=0`)
- 在 `start_main()` 中调用: `fsinit(ROOTDEV)` (`ROOTDEV=1`, 表示 `virtio` 磁盘设备)

6.3 `iget(uint dev, uint inum)` — “给我这个 inode 的内存引用”

设想你要打开 `/etc/passwd`, 内核找到了它对应的 inode 编号 (`inum=7`)。调用

`iget(dev, 7)` 就是说: “给我 inode 7 的内存缓存项, 我要用它。”

关键点: `iget` 不从磁盘读取内容 (这是 `ilock` 的事)。它只是在 `icache` 中找到或分配一个槽位, 递增引用计数, 然后返回。

算法:

```
1 扫描 icache.inode[0..NINODE-1]:
2    分支A: 找到 dev==dev && inum==inum && ref>0 的条目
3        → 引用计数+1 (ref++), 返回该 inode 指针
4        → (注意: 不重置 valid! 已有缓存的继续使用)
5    分支B: 同时记录第一个 ref==0 的空槽 (empty)
6
7 扫描结束后, 若未命中且有空槽 (empty != 0):
8    → 用这个空槽: fill in dev, inum; ref=1; valid=0; 返回
9 若既没命中也没空槽:
10    → panic("iget: no inodes")
```

6.4 ⚠ 理论和实践的第六道鸿沟: `iget` 不读磁盘, `ilock` 才读磁盘

这是整个文件系统最容易出错的地方之一。

常见误解: `iget` 应该从磁盘把 inode 数据读进来。

正确理解: `iget` 只在 inode 缓存 (`icache`) 中操作——找一个空闲槽或复用已有槽, 把 `ref` 加 1, 然后返回。它永远不会触发磁盘 I/O。

为什么要这样设计? 因为 inode 缓存的目的是减少磁盘 I/O。如果每次调用 `iget` 都读磁盘, 那缓存就毫无意义了。

两个函数的分工:

- `iget`: 在缓存中获取或分配一个条目, **不读磁盘**
- `ilock`: 在 `iget` 之后调用, **才真正从磁盘读取 inode 数据**

⚠ **常见错误:** 学生在实现中让 `iget` 做了 `ilock` 的工作 (直接调用 `bread` 读磁盘)。

这会导致:

1. 同一个 inode 被重复从磁盘读取 (性能灾难)
2. 破坏 `icache` 的一致性假设 (当 inode 已在缓存中时, `iget` 不应刷新它)

正确顺序:

```
1 ip = iget(dev, inum);    // 只找缓存槽, valid 可能为 0
2 ilock(ip);               // 现在才从磁盘加载数据, valid 变为 1
3 // ... 使用 inode ...
4 iunlock(ip);
5 iput(ip);
```

参考 xv6 `kernel/fs.c` 的 `iget` 函数，注意它是如何用两个变量（一个记“已命中”，一个记“备用空槽”）在单次遍历中完成这件事的。

6.5 `ilock(struct inode *ip)` — “我要访问它的内容，先给我把数据加载进来”

`iget` 返回的 `inode` 可能 `valid==0`（数据还在磁盘上，没加载）。`ilock` 负责：若 `valid==0`，从磁盘读取对应的 `struct dinode`，把字段复制到内存 `inode`。

实现步骤：

```
1 void ilock(struct inode *ip) {
2     if(ip->valid == 0) {
3         // 计算这个 inode 在哪个磁盘块
4         uint blockno = sb.inodestart + ip->inum / IPB;
5         struct buf *bp = bread(ip->dev, blockno);
6         // 找到块内偏移处的 dinode
7         struct dinode *dip = (struct dinode*)bp->data + ip->inum % IPB;
8         // 把磁盘 dinode 的字段复制到内存 inode
9         ip->type = dip->type;
10        ip->major = dip->major;
11        ip->minor = dip->minor;
12        ip->nlink = dip->nlink;
13        ip->size = dip->size;
14        memmove(ip->addrs, dip->addrs, sizeof(ip->addrs));
15        brelse(bp);
16        ip->valid = 1;
17    }
18    if(ip->type == 0)
19        panic("ilock: no type");
20 }
```

简化说明：本实验可以不实现真正的睡眠锁（`sleeplock`），`ilock` / `iunlock` 只做磁盘加载，并发安全留作进阶任务。

参考 xv6 `kernel/fs.c` 的 `ilock`，注意它在加锁前后对 `valid` 的检查模式。

6.6 `iunlock(struct inode *ip)` — 对应 `ilock` 的解锁

简化版中，是一个空函数或只做检查（`ip->ref >= 1`）。若你实现了睡眠锁，则在这里释放锁。

6.7 `iput(struct inode *ip)` — “我用完了，还给你”

调用 `iput` 表示当前代码不再需要这个 `inode` 的指针。`ref--`，若归零，说明没有人再持有这个 `inode`：

```
1 void iput(struct inode *ip) {
2     ip->ref--;
```

```

3 // 若引用归零且 nlink==0 (文件已被 unlink), 应当释放所有数据块
4 // 简化版: 先只做 ref 管理, 不处理数据块释放
5 if(ip->ref == 0) {
6     ip->valid = 0; // 标记缓存槽空闲, 供下次 iget 重用
7 }
8 }

```

参考 xv6 `kernel/fs.c` 的 `iput`, 理解它在 `ref==0 && nlink==0` 时如何调用 `itrunc` 释放所有数据块。本实验可以先简化: 只做引用计数管理。

6.8 `iupdate(struct inode *ip)` — “把内存里的修改写回磁盘”

当你修改了 inode (比如 `writei` 之后更新了 `ip->size`), 这个修改只在内存中。

`iupdate` 将其同步写回磁盘的 inode 区:

```

1 void iupdate(struct inode *ip) {
2     struct buf *bp = bread(ip->dev, sb.inodestart + ip->inum / IPB);
3     struct dinode *dip = (struct dinode*)bp->data + ip->inum % IPB;
4     // 把内存字段写回磁盘 dinode (与 ilock 的方向相反)
5     dip->type = ip->type;
6     dip->major = ip->major;
7     dip->minor = ip->minor;
8     dip->nlink = ip->nlink;
9     dip->size = ip->size;
10    memmove(dip->addrs, ip->addrs, sizeof(ip->addrs));
11    bwrite(bp);
12    brelse(bp);
13 }

```

6.9 `ialloc(uint dev, short type)` — “在磁盘上给我分配一个新 inode”

创建文件时需要分配 inode。`ialloc` 遍历磁盘 inode 区, 找到第一个 `type==0` 的 `dinode`, 将其类型设置并写回磁盘, 然后返回其内存缓存项:

```

1 struct inode *ialloc(uint dev, short type) {
2     for(int inum = 1; inum < sb.ninodes; inum++) {
3         struct buf *bp = bread(dev, sb.inodestart + inum/IPB);
4         struct dinode *dip = (struct dinode*)bp->data + inum%IPB;
5         if(dip->type == 0) { // 找到空闲 inode
6             memset(dip, 0, sizeof(*dip));
7             dip->type = type; // 占用并设置类型
8             bwrite(bp);
9             brelse(bp);
10            return iget(dev, inum); // 返回对应内存 inode
11        }
12        brelse(bp);
13    }

```

```

14     panic("ialloc: no inodes");
15 }

```

参考 xv6 `kernel/fs.c` 的 `ialloc`，结构完全相同。

第七节：块映射与文件读写——框架 TODO + 自主设计

7.1 `bmap(ip, bn)` — 逻辑块号翻译成物理块号（框架 TODO）

一个文件的第 `bn` 个逻辑块，实际存在磁盘的哪个物理块？这就是 `bmap` 的工作：

```

1  bn < NDIRECT(12) :    → 直接查 ip->addrs[bn]
2  bn < NDIRECT + 256 : → 先找间接块 ip->addrs[NDIRECT]，再从那个块里找第 bn-NDIR

```

框架 TODO 步骤1：若 `ip->addrs[bn] == 0`，说明这个逻辑块还没有分配物理块，调用 `balloc(ip->dev)` 分配一个，写入 `ip->addrs[bn]`。

框架 TODO 步骤2：若 `ip->addrs[NDIRECT] == 0`，间接指针块本身也没分配，先 `balloc` 分配并写入 `ip->addrs[NDIRECT]`。

框架 TODO 步骤3：读出间接块（`bread(ip->dev, ip->addrs[NDIRECT])`），在其中找 `a[bn] == 0` 的项，`balloc` 分配后写入 `a[bn]`，然后必须 `bwrite(bp)` 将间接块写回磁盘（否则间接块的修改丢失），最后 `brelease(bp)`。

7.2 `readi(ip, dst, off, n)` — 从 inode 读数据（框架 TODO）

框架已有循环骨架，只缺最后的 `memcpy`（TODO 步骤4）：

```

1  // 循环体内，bp = bread(ip->dev, bmap(ip, off/BSIZE)) 之后：
2  m = min(BSIZE - off%BSIZE, n - tot);    // 本次能读多少字节
3  memmove((void*)dst, bp->data + off%BSIZE, m);

```

注意 `off%BSIZE` 是块内偏移——文件字节偏移 `off` 对应的块内位置。

7.3 `writei(ip, src, off, n)` — 向 inode 写数据（自主设计）

`writei` 是 `readi` 的“镜像”——方向相反。参考 `readi` 的循环结构实现：

参考 xv6：阅读 `xv6-riscv/kernel/fs.c` 中的 `writei` 函数（约40行）。

关键差异（与 `readi` 的不同点）：

1. 拷贝方向反过来：`memmove(bp->data + off%BSIZE, src, m)`（从 `src` 拷到磁盘块）
2. 修改缓冲块后必须 `bwrite(bp)` 写回磁盘
3. 写入可能使文件变大：若 `off + n > ip->size`，则更新 `ip->size = off + n`
4. 无论文件大小是否变化，结束后必须 `iupdate(ip)` 将新的 `size` 写回磁盘的 inode

函数签名：

```

1  int writei(struct inode *ip, int user_src, uint64 src, uint off, uint n)
2  // 简化版：user_src 始终为 0（内核地址），直接 memmove 即可

```

边界检查：

- `off > MAXFILE * BSIZE`：偏移超出最大文件大小，返回 -1
- 若写入后 `off + n > MAXFILE * BSIZE`，截断 `n` 到最大允许值

7.4 ⚠ 理论和实践的第五道鸿沟：写文件是“整块写入”，不是“字节追加”

教科书讨论文件 I/O 时，经常说“向文件追加数据”，让学生以为 `write` 系统调用是一条神奇的指令，数据自动追加到文件末尾。

实际上，写入流程是：

1. 根据当前文件偏移 `off`，计算目标逻辑块号 `bn = off / BSIZE`
2. 通过 `bmap(ip, bn)` 找到或分配物理块
3. 通过 `bread` 把整个物理块读入内存（如果尚未在缓存中）
4. 在内存中修改块内对应位置的数据
5. 调用 `bwrite` 把整个块写回磁盘

这意味着：如果你要写文件的前 5 个字节，而文件的第 0 块已经在缓存中，`bmap` 不会分配新块——你会直接修改第 0 块的字节 0~4，然后写回。

也意味着：如果你写了跨越块边界的数据（比如从字节 1020 开始写 100 字节），你需要修改第 0 块的后 4 字节，再修改第 1 块的前 96 字节，需要两次 `bread` / `bwrite` 操作。

这是 `writeti` 用循环而不是单次操作的原因——循环处理了跨块写入的情况。

第八节：目录与路径——框架 TODO + 自主设计

7.1 “目录也是文件”——这句话是什么意思？

对内核而言，目录 = 类型为 `T_DIR` 的 inode + 内容为 `struct dirent[]` 数组的数据块。
每个 `struct dirent` 是一条目录记录：

```
1 struct dirent {
2     ushort inum;        // 指向的 inode 编号 (0 表示空槽)
3     char    name[14];    // 文件名 (最多14字符)
4 };                      // 共16字节
```

目录的“内容”就是这种记录的列表。读目录 = 用 `readi` 从目录 inode 读出 `struct dirent` 数组。所以 `dirlookup` 本质上是：用 `readi` 遍历目录内容，比较每条记录的名字字段。

7.2 `dirlookup(dp, name, poff)` ——在目录中查名字（框架 TODO）

框架 TODO 步骤：在已有的循环中（框架已提供），你需要：

1. 实现字符串比较（最多 `DIRSIZ` 个字符）：不能用标准库的 `strcmp`，需要自己写一个简单的循环比较
2. 若名字匹配：
 - 若 `poff != 0`，写入 `*poff = off`
 - 将 `de.inum` 赋给局部变量 `inum`
 - 调用 `iget(dp->dev, inum)` 并返回

自定义字符串比较（参考思路）：

```

1 // 比较 a 和 b 的前 n 个字符，相等返回 0
2 int mystrncmp(const char *a, const char *b, int n) {
3     for(int i = 0; i < n; i++) {
4         if(a[i] != b[i]) return 1;
5         if(a[i] == 0) return 0; // 两者同时到达结尾
6     }
7     return 0;
8 }

```

7.3 ⚠ 理论和实践的第七道鸿沟：目录项的 name 为什么是 14 字节？

教科书上讲目录项时，通常会说“目录项包含文件名和 inode 编号”。但很少有人问：为什么文件名长度是 14 字节而不是 16 或 32？

答案是：这是磁盘空间利用率和功能性的权衡。

每个 `struct dirent` 是 16 字节：`inum` 占 2 字节，`name` 占 14 字节。如果 `name` 是 16 字节，`struct dirent` 就会变成 18 字节（由于对齐可能还是 20 字节）。这会导致：

1. 磁盘空间浪费（每个目录项多了 2~4 字节）
2. 目录遍历时代码更复杂（需要处理不对齐的边界）

xv6 选择 14 字节是在“足够存放下大多数文件名”和“节省磁盘空间”之间的折中。14 字节限制意味着你可以创建 `README.txt` 这样的文件，但不能创建超长文件名。

实践中，当你在 `dirlookup` 中比较 `de.name` 和 `name` 时，比较的是前 `DIRSIZ` 个字节，而不是遇到 `\0` 就停止。这是因为 `name[14]` 数组中未使用的部分是未定义的（可能是 0，也可能是其他垃圾值）。所以比较时必须用固定的比较长度：

```

1 // 错误：遇到 \0 就停止，可能误匹配
2 if(strcmp(de.name, name) == 0) // 不要这样写！
3
4 // 正确：固定比较前 DIRSIZ 个字节
5 if(memcmp(de.name, name, DIRSIZ) == 0) // 这样写

```

7.4 `namei(path)` — 把路径字符串变成 inode（自主设计）

这是文件系统面向用户的“导航员”。用户说 `open("/etc/passwd", ...)` —— `namei` 的任务就是：一步一步地走这条路径，最终返回 `passwd` 对应的 inode。

参考 xv6：阅读 `xv6-riscv/kernel/fs.c` 中的 `namex`、`namei`、`nameiparent` 三个函数（约 50 行），这是整个文件系统最值得细读的部分之一。

你需要实现的辅助函数 `skipelem(path, name)`：

```

1 skipelem("/etc/passwd", name) → 填入 name="etc", 返回 "/passwd"
2 skipelem("/passwd", name)     → 填入 name="passwd", 返回 ""
3 skipelem("", name)            → 返回 0（路径结束）

```

实现逻辑：跳过开头的 `/`，把下一个 `/` 之前的内容拷到 `name`，返回 `/` 之后的剩余路径。

`namei(path)` 的工作流程：

```
1 1. 若 path[0]=='/' : 从根目录开始 → ip = iget(ROOTDEV, ROOTINO)
2 否则 : 从当前进程工作目录开始 (简化版 : 始终用根目录)
3
4 2. 循环 :
5   a. path = skipelem(path, name) // 取下一个路径分量
6   b. 若 path == 0 : 说明已到终点, 返回当前 ip (即目标文件的 inode)
7   c. ilock(ip) // 准备读取目录内容
8   d. 若 ip->type != T_DIR : 文件不是目录, iunlock + iput, 返回 0
9   e. next = dirlookup(ip, name, 0) // 在当前目录中查 name
10  f. iunlock + iput(ip)
11  g. 若 next == 0 : 文件不存在, 返回 0
12  h. ip = next, 继续下一轮
```

7.5 ⚠ 理论和实践的第八道鸿沟：skipelem 的三个边界条件

skipelem 的实现比大多数学生想象的复杂，因为路径字符串有三种完全不同的形式：

输入	期望输出
<code>"/etc/passwd"</code>	<code>name="etc"</code> , 返回 <code>"/passwd"</code>
<code>"/passwd"</code>	<code>name="passwd"</code> , 返回 <code>""</code>
<code>""</code>	返回 <code>0</code> (空路径)

最容易犯的错误是只处理了第一种情况，忽略了后面两种。

另外还有两个边界：

- 连续斜杠 `"/usr//local"`：第二个分量是空字符串，应该跳过
- 尾部斜杠 `"/etc/"`：最后返回空字符串，但 `namei` 应该把它视为有效路径（走到 `etc` 目录）

7.6 `dirlink(dp, name, inum)` — 在目录中添加一条记录（必做）

`dirlink` 是 `sys_open(O_CREAT)` 创建新文件时的关键一步——当 `ialloc` 分配了新 inode 后，必须调用 `dirlink` 在父目录的数据块中写入一条 `struct dirent` 记录。

实现思路：

```
1 在父目录的数据块中找一个空槽 (de.inum == 0)，或追加到目录末尾：
2   1. 遍历目录文件的每一个 dirent 槽位 (off 从 0 到 dp->size)
3   2. 找到第一个 de.inum == 0 的空槽
4   3. 将 de.inum = inum, de.name = name (注意用 memset 清零后复制名字)
5   4. 调用 writei() 将这个 dirent 写回磁盘
```

参考 xv6: 阅读 `xv6-riscv/kernel/fs.c` 中的 `dirlink` 函数（约20行）。

⚠️ 常见错误: 忘记用 `memset` 先清零 `de.name` 数组的剩余空间, 导致残留垃圾数据与 `dirlookup` 的 `memcmp` 行为不一致。

第九节：文件描述符层——自主设计

8.1 为什么需要文件描述符？

用户进程打开文件后, 不是直接拿着 inode 操作——它们拿的是一个整数（文件描述符, `fd`）。这个整数背后藏着：

- 当前读写的位置偏移（读到哪了？写到哪了？）
- 这个文件是只读还是只写
- 指向对应 inode 的指针

为什么要这一层间接？若两个进程打开同一文件, 它们可以有各自的偏移（不互相干扰）, 但共享同一个 inode（共享文件内容）。`fork()` 后父子进程还能共享 `fd`——这些都需要通过文件描述符层来管理。

8.2 ⚠️ 理论和实践的第九道鸿沟：fd 不是 inode 编号的别名

很多学生（包括 xv6 最初的设计者）会问：为什么不能直接用 `inode` 编号作为文件描述符？为什么要多一层 `struct file`？

答案是：inode 编号是磁盘上文件的身份标识, 而文件描述符是进程打开文件的实例。这两者有本质区别：

1. 同一文件可以被一个进程打开多次, 每次得到不同的 `fd`, 但都指向同一个 inode
2. 两个进程可以各自独立地读写同一文件, 各自维护自己的偏移量
3. `fork()` 后父子进程共享同一个 `fd`, 但各自有自己的 `struct file` 副本吗？不对——它们共享同一个 `struct file`（通过 ref 计数）, 所以共享同一个偏移量

省略 `struct file` 层意味着什么？如果 `fd` 直接对应 inode：

- 进程 A 读取 10 字节后, 进程 B 读取时不知道从哪开始（inode 没有偏移字段）
- `fork()` 的子进程无法独立偏移（会互相覆盖）

8.3 数据结构设计

参考 xv6: 阅读 `xv6-riscv/kernel/file.h` 和 `xv6-riscv/kernel/file.c`。

你需要在新文件 `kernel/fs/file.c` 中设计：

```
1 // 文件对象（一个打开文件的实例）
2 struct file {
3     int    type;           // FD_NONE=0, FD_INODE=1（本实验先只支持 inode 文件）
4     int    ref;           // 引用计数（fork 后父子共享时 ref==2）
5     char   readable;      // 是否可读
6     char   writable;      // 是否可写
7     struct inode *ip;     // 对应的 inode（type==FD_INODE 时有效）
8     uint   off;           // 当前读写偏移（字节）
```



```

9  };
10
11 // 全局文件表（系统中所有打开的文件对象）
12 struct {
13     struct file file[NFILE]; // NFILE 在 param.h 中定义（如64）
14 } ftable;

```

进程的文件描述符表（**必须在** `proc.h` 的 `struct proc` 中添加，这是整个文件描述符层的前提）：

在 `proc.h` 的 `struct proc` 定义中，找到 `char name[16];` 那一行，在其下方添加：

```

1  struct proc {
2      enum task_status status;
3      int pid;
4      pagetable_t pagetable;
5      struct trapframe *trapframe;
6      struct context context;
7      uint64 kstack;
8      uint64 sz;
9      char name[16];
10     struct file *ofile[NFILE]; // 进程打开的文件表, fd=3 → ofile[3]
11 };

```

同时在 `kernel/include/param.h` 中确认 `NFILE` 已定义（通常为 16 或 64）。如果未定义，需要添加 `#define NFILE 16`。

8.3 新建文件 `kernel/fs/file.c`（骨架框架）

本文件需要**从零创建**。你需要先阅读 xv6 的 `file.c` 和 `file.h` 理解设计思路，然后自行建模实现。

你需要创建的内容框架：

```

1  /* file.c – 文件描述符层（Lab7 任务）
2   *
3   * 本文件从零创建。
4   * 参考 xv6-riscv/kernel/file.c 和 kernel/file.h。
5   */
6
7  #include "defs.h"
8  #include "param.h"
9  #include "proc.h"
10 #include "types.h"
11
12 /* 全局文件表 */
13 struct {
14     struct file file[NFILE];
15 } ftable;
16
17 /* 你需要实现的数据结构（struct file）已在上节描述。

```

```

18  * 请在 kernel/include/file.h 中声明，或直接在此文件顶部声明。
19  */
20
21  /* =====
22  * filealloc - 从文件表分配一个空闲 file 对象
23  *
24  * 算法：遍历 ftable.file[]，找 ref==0 的槽位，ref=1，返回该 file 指针
25  * TODO：自行实现
26  * ===== */
27
28  /* =====
29  * filedup - 复制一个 file (fork 时使用)
30  *
31  * TODO：自行实现（增加 ref 计数，返回同一 file 指针）
32  * ===== */
33
34  /* =====
35  * fileclose - 关闭文件，释放引用
36  *
37  * 算法：
38  *   ref--；若 ref 归零：
39  *     - 调用 iput(f->ip) 释放 inode 引用
40  *     - 清零 file 结构 (type=0, ip=0, off=0 等)
41  * TODO：自行实现
42  * ===== */
43
44  /* =====
45  * fileread - 从文件读取数据
46  *
47  * 算法：
48  *   - 检查 f->readable
49  *   - ilock(f->ip)
50  *   - 调用 readi() 读取
51  *   - f->off += 实际读取字节数
52  *   - iunlock(f->ip)
53  *   - 返回读取字节数（或错误码 -1）
54  * TODO：自行实现
55  * ===== */
56
57  /* =====
58  * filewrite - 向文件写入数据
59  *
60  * TODO：自行实现（参考 fileread，调用 writei 而非 readi）
61  * ===== */

```

8.4 新建文件 `kernel/syscall/sysfile.c`（骨架框架）

本文件需要从零创建。你需要先阅读 xv6 的 `sysfile.c` 理解设计思路，然后自行建模实现。

你需要创建的内容框架：

```
1  /* sysfile.c - 文件相关系统调用 (Lab7 任务)
2
3  * 本文件从零创建。
4  * 参考 xv6-riscv/kernel/sysfile.c, 理解 sys_open/sys_read/sys_write/sys_c
5  */
6
7  #include "defs.h"
8  #include "param.h"
9  #include "proc.h"
10 #include "types.h"
11
12 /* O_CREAT、O_RDONLY、O_WRONLY 等 flag 常量
13  * 在此定义 (或从其他头文件引用) :
14  */
15 #ifndef O_CREAT
16 #define O_CREAT  0x001
17 #endif
18 #ifndef O_WRONLY
19 #define O_WRONLY 0x002
20 #endif
21 #ifndef O_RDONLY
22 #define O_RDONLY 0x000
23 #endif
24
25 /* =====
26  * sys_open - 打开或创建文件
27  *
28  * 算法 (参考 xv6 sys_open 实现思路) :
29  *   1. 从 trapframe 读取 path (用户态虚拟地址) 和 flags
30  *   2. 调用 copyinstr() 将用户态 path 复制到内核缓冲区
31  *   3. 若 O_CREAT 标志设置 :
32  *       - 调用 nameiparent() 分离父目录路径和文件名
33  *       - 在父目录中 dirlookup() 查找是否已存在
34  *       - 若不存在 : ialloc() 分配新 inode, dirlink() 在目录中注册
35  *       - 若存在 : 直接使用
36  *   4. 若无 O_CREAT : 调用 namei() 直接查找文件
37  *   5. filealloc() 分配 file 对象
38  *   6. 在 proc.ofile[] 中找空闲槽位, 记录 fd
39  *   7. 初始化 file 结构 (type=FD_INODE, ip, off, readable, writable)
40  *   8. 返回 fd
41  *
42  * TODO : 自行实现上述逻辑
43  * ===== */
44
45 /* =====
46  * sys_read - 读取文件
47  *
```

```

48 * 算法：
49 *   1. 从 trapframe 读取 fd、addr、n
50 *   2. 检查 fd 合法性 (p->ofile[fd] != 0)
51 *   3. 调用 fileread() 读取数据
52 *   4. 返回实际读取字节数
53 *
54 * TODO：自行实现
55 * ===== */
56
57 /* =====
58 * sys_write - 写入文件
59 *
60 * TODO：自行实现（参考 sys_read 的结构）
61 * ===== */
62
63 /* =====
64 * sys_close - 关闭文件
65 *
66 * TODO：自行实现
67 * ===== */
68
69 /* 辅助函数（你需要实现或找到现有实现）：
70 * copyinstr(user_src, char *dst, int max) - 从用户虚拟地址复制字符串到内核
71 *   简化版：本框架用户态和内核态共用页表，可直接 memmove
72 *   完整版：参考 xv6 的 copyinstr() / copyin()
73 */

```

`sys_read(fd, addr, n)` 的实现骨架：

```

1  /* =====
2  * sys_read - 读取文件
3  *
4  * 算法：
5  *   1. 从 trapframe 读取 fd、addr、n
6  *   2. 检查 fd 合法性 (p->ofile[fd] != 0)
7  *   3. 调用 fileread() 读取数据
8  *   4. 返回实际读取字节数
9  *
10 * TODO：自行实现
11 * ===== */
12
13 /* =====
14 * sys_write - 写入文件
15 *
16 * TODO：自行实现（参考 sys_read 的结构）
17 * ===== */
18
19 /* =====

```

```

20 * sys_close - 关闭文件
21 *
22 * TODO: 自行实现
23 * ===== */
24
25 /* 辅助函数（你需要实现或找到现有实现）：
26 * copyinstr(user_src, char *dst, int max) - 从用户虚拟地址复制字符串到内核
27 * 简化版：本框架用户态和内核态共用页表，可直接 memmove
28 * 完整版：参考 xv6 的 copyinstr() / copyin()
29 */

```

第十节：把所有东西连接起来——初始化与启动顺序

9.1 `start_main()` 最终形态

```

1 void start_main() {
2     kinit();           // Lab3: 物理内存分配器
3     kvmininit();       // Lab3: 内核页表（恒等映射）
4     kvminithart();     // Lab3: 激活分页
5     trapinithart();    // Lab4: 注册中断向量
6     intr_on();         // Lab4: 开启 S-Mode 中断
7     procinit();        // Lab5: 初始化进程表
8     fsinit(ROOTDEV);   // Lab7: ← 初始化文件系统（先 binit, 再读超级块）
9     userinit();        // Lab5: 创建 proczero 用户进程
10    scheduler();        // Lab5: 进入调度器（永不返回）
11 }

```

9.2 需要自行定义的常量和枚举

框架的 `param.h` 和 `types.h` 中没有以下常量，你需要在 `kernel/include/types.h` 或 `kernel/include/param.h` 中添加，或在具体实现文件中 `static` 定义：

```

1 /* ===== 文件系统魔数 ===== */
2 #define FSMAGIC 0x10203040 // xv6 文件系统魔数（用于验证文件系统有效性）
3
4 /* ===== 根目录 inode 编号 ===== */
5 #define ROOTINO 1 // 根目录的 inode 编号（固定为 1）
6
7 /* ===== inode type（文件类型） ===== */
8 #define T_FILE 1 // 普通文件
9 #define T_DIR 2 // 目录文件
10 #define T_DEVICE 3 // 设备文件
11
12 /* ===== file type（文件描述符类型） ===== */
13 #define FD_NONE 0 // 未使用

```

```
14 #define FD_INODE 1 // inode 文件（本实验只支持这一种）
```

注意：框架 `kernel/include/defs.h` 中已有 `struct buf; struct inode; struct dirent;` 的前向声明，但没有 `struct file` 和 `struct superblock` 的前向声明。你需要在 `fs.c` 或 `file.c` 中自行声明（或直接定义结构体）。

9.3 你的代码应该放在哪个文件

函数	建议放在
<code>binit</code> / <code>bget</code> / <code>bread</code> / <code>bwrite</code> / <code>brelease</code>	<code>kernel/fs/bio.c</code> (框架已有, 补全 TODO)
<code>bzero</code> / <code>balloc</code> / <code>bfree</code>	<code>kernel/fs/fs.c</code> (在 <code>bmap</code> 之前)
<code>fsinit</code> / <code>iget</code> / <code>ilock</code> / <code>iunlock</code> / <code>iput</code> / <code>iupdate</code> / <code>ialloc</code>	<code>kernel/fs/fs.c</code>
<code>bmap</code> / <code>readi</code> / <code>writei</code>	<code>kernel/fs/fs.c</code> (<code>bmap</code> / <code>readi</code> 框架已有 TODO)
<code>dirlookup</code> / <code>dirlink</code> / <code>skipelem</code> / <code>namei</code> / <code>nameiparent</code>	<code>kernel/fs/fs.c</code>
<code>filealloc</code> /	<code>kernel/fs/file.c</code> (新建)

<pre> / fileclose / fileread / filewrite </pre>	(新建)
<pre> sys_open / sys_read / sys_write / sys_close </pre>	<pre>kernel/syscall/sysfile.c</pre> (新建或扩展)

不要忘记在 `kernel/include/defs.h` 中添加所有新函数的声明。

9.4 `defs.h` 新函数声明清单

框架已有声明：`bio.c` 的 `binit/bread/bwrite/brelse` 和 `fs.c` 的 `fsinit/iget/dirlookup/readi/writel` 已在框架的 `defs.h` 中声明，无需重复添加。

以下是你需要额外添加的函数声明和结构体前向声明：

```

1 // ===== 添加结构体前向声明 =====
2 struct file;    // 框架 defs.h 中已有 buf/inode/dirent, 需添加 file
3
4 // ===== fs.c (文件系统核心) =====
5 // 以下函数需自行添加到 defs.h:
6 struct inode *ialloc(uint dev, short type);
7 void ilock(struct inode *ip);
8 void iunlock(struct inode *ip);
9 void iput(struct inode *ip);
10 void iupdate(struct inode *ip);
11 uint bmap(struct inode *ip, uint bn);
12 int dirlink(struct inode *dp, char *name, uint inum);
13 struct inode *namei(char *path);
14 struct inode *nameiparent(char *path, char *name);
15 // balloc/bfree/bzero 为 static 函数, 仅限 fs.c 内部使用, 无需在 defs.h 声明
16
17 // ===== file.c (文件描述符层) =====
18 // 全部需要添加:
19 struct file *filealloc(void);
20 struct file *filedup(struct file *f);
21 void fileclose(struct file *f);
22 int fileread(struct file *f, uint64 addr, int n);
23 int filewrite(struct file *f, uint64 addr, int n);

```

```

24
25 // ===== sysfile.c (系统调用) =====
26 // 全部需要添加：
27 uint64 sys_open(void);
28 uint64 sys_read(void);
29 uint64 sys_write(void);
30 uint64 sys_close(void);

```

同时别忘了在 `syscall.c` 的 `syscalls[]` 表中注册文件相关系统调用：

注意：系统调用号必须与用户态库中使用的编号一致。请确认用户程序中

`open/read/write/close` 的系统调用号，或自行在 `syscall.c` 头部定义一致的编号。

```

1 // 在 syscall.c 的 syscalls[] 数组中添加（参考 Lab6 的 sys_write）：
2 // 定义 syscall 号（确保与用户态保持一致）
3 #define SYS_open 15 // 或其他未使用的编号
4 #define SYS_read 16 // 需与用户态open/read/write/close的编号对应
5 #define SYS_close 17
6 // ...
7 [SYS_open] sys_open,
8 [SYS_read] sys_read,
9 [SYS_close] sys_close,

```

第十一节：验收测试

在 `proczero` 中添加以下测试（你需要先实现 Lab6 的系统调用才能调用）：

```

1 // === 测试1：创建并写文件 ===
2 int fd = open("/hello.txt", O_CREAT | O_WRONLY);
3 if(fd < 0) { printf("open failed\n"); goto fail; }
4 int r = write(fd, "Hello, File System!\n", 20);
5 if(r != 20) { printf("write failed: %d\n", r); goto fail; }
6 close(fd);
7 printf("Test 1 passed: write %d bytes\n", r);
8
9 // === 测试2：读回文件内容 ===
10 fd = open("/hello.txt", O_RDONLY);
11 if(fd < 0) { printf("reopen failed\n"); goto fail; }
12 char buf[64] = {0};
13 r = read(fd, buf, 64);
14 close(fd);
15 if(r != 20) { printf("read wrong length: %d\n", r); goto fail; }
16 printf("Test 2 passed: read back: %s", buf); // 应输出 "Hello, File Syst

```

期望输出：

```
1 Test 1 passed: write 20 bytes
2 Test 2 passed: read back: Hello, File System!
```

第十二节：分阶段调试策略

不要等全部实现完再运行，每完成一个阶段就验证：

完成阶段	验证方法
binit + bread / brelse	fsinit 中读超级块, GDB 检查 sb.magic == 0x10203040
balloc	调用两次 balloc , 检查返回的块号不同且不为0
iget + ilock	iget(1, 1) 获取根目录 inode, ilock 后检查 ip->type == 2 (目录)
bmap	对一个空文件调用 bmap(ip, 0) , 检查返回的块号非0, 且位图对应 bit 变为1
readi	通过 readi 读根目录 (inode 1) 前16字节, 应能解析出 . 目录项 (inum=1)
dirlookup	dirlookup(root_inode, ".", 0) 应返回根目录 inode 自身
writei	写入后 readi 读回, 比较内容; 也检查 ip->size 是否正确更新
namei	namei("/") 返回根目录 inode; namei("/nonexistent") 返回0
系统调用	运行验收测试, 对比输出

第十三节：常见错误排查

症状	最可能的原因
<code>"invalid file system"</code> panic	磁盘镜像未生成（检查 <code>Makefile</code> 是否执行了 <code>mkfs</code> ），或超级块偏移计算错误
<code>"bget: no buffers"</code> panic	<code>brelse</code> 没有正确将 <code>refcnt</code> 归零，缓冲块从未回收
<code>bmap</code> 返回0或panic	<code>bmap</code> TODO 步骤1忘记 <code>addr = ip->addrs[bn]</code> (只分配了，没赋值给 <code>addr</code>)
写入后读回数据是全零	<code>bmap</code> 步骤3修改了间接块但忘记 <code>bwrite(bp)</code>
<code>writei</code> 后文件大小没变	忘记更新 <code>ip->size</code> 或忘记调用 <code>iupdate(ip)</code>
<code>dirlookup</code> 永远返回0	<code>readi</code> 步骤4的 <code>memcpy</code> 未实现， <code>de.name</code> 始终是空数据
<code>namei("/usr/README")</code> panic	<code>skipelem</code> 边界处理错误：路径以 <code>/</code> 结尾、或连续 <code>//</code> 情况
<code>open</code> 系统调用返回 <code>-1</code>	没有在 <code>ofile[]</code> 中找到空槽，或 <code>filealloc</code> 返回0（文件表满）

第十四节：思考题（实验报告必答）

1. 关于 LRU 不变式：`brelse` 把刚释放的块移到链表头部，`bget` 从链表尾部往前找淘汰候选。这个设计保证了“最近使用的块最后被淘汰”——请用一个具体的例子（假设缓冲池共 3 个槽位，依次操作块 A→B→C→A→D）手动模拟链表状态的变化，验证这个性质。
2. 关于 `iget` 的“不读磁盘”设计：`iget` 只分配缓存槽（`valid=0`），`ilock` 才读磁盘（设置 `valid=1`）。为什么要分成两步？如果 `iget` 直接读磁盘，会有什么问题？（提示：考虑两个进程同时 `iget` 同一个 inode 的情况）
3. 关于目录是文件：`dirlookup` 内部调用了 `readi`。请描述当你 `dirlookup(root_inode, "etc", 0)` 时，数据是如何从磁盘流到比较逻辑的——要求说出调用链中每个函数的作用。
4. 关于文件描述符的两层设计：为什么不直接让文件描述符 `fd`（整数）直接对应 inode 编号，而是要通过 `struct file` 作为中间层？举一个如果省略 `struct file`（`fd` 直接对应 inode）会出问题的场景。
5. 关于持久性：在 `writel` 中，你调用了 `bwrite(bp)` 将修改写回磁盘。如果省略这个 `bwrite`（只修改内存中的 `buf->data`，不写磁盘），系统会立刻崩溃吗？数据会丢失吗？在什么情况下可以不立刻 `bwrite`，而是等到 `brelse` 的时候再写？（提示：想想 xv6 的日志机制）

第十五节 进阶挑战（可选）

- 进阶1：实现 `unlink` 系统调用（减少 `nlink`，若 `nlink==0` 则 `bfree` 释放所有数据块）

进阶2 — 实现日志层（崩溃恢复）

参考 xv6 的 `log.c`，实现写前日志（Write-Ahead Log）。核心接口：`begin_op()` / `log_write()` / `end_op()`。使用日志后，所有磁盘写操作具有原子性，系统断电重启后能从日志恢复。

进阶3 — 支持二级间接块

目前 `bmap` 只实现了直接块（12块）和一级间接块（256块），最大文件为 $(12+256)*1024 \approx 268\text{KB}$ 。扩展 `bmap` 支持二级间接块，将最大文件大小提升至约 65MB。

进阶4 — 实现并发安全的 inode 锁

当前简化版中 `ilock` / `iunlock` 不实现真正的互斥。在 Lab5 已实现多进程的基础上，使用自旋锁或睡眠锁保护 inode 缓存，使文件系统支持真正的多进程并发访问。

第十六节 实验完成后的系统全貌

完成本实验后，`start_main()` 的初始化序列如下：

```
1 void start_main() {
2     kinit();           // Lab3 : 内存分配器
3     kvmininit();       // Lab3 : 内核页表
4     kvminithart();     // Lab3 : 开启分页
5     trapinithart();    // Lab4 : 陷阱向量
6     intr_on();         // Lab4 : 开启中断
7     procinit();        // Lab5 : 进程表
8     fsinit(ROOTDEV);   // Lab7 : 文件系统（含 binit + 超级块加载）
9     userinit();        // Lab5 : 创建 proczero
```

```
10     scheduler();        // Lab5 : 进入调度器
11 }
```

至此，你已构建了一个从裸机启动到能够持久化存储数据的完整操作系统原型：

硬件初始化 → 内存管理 → 虚拟地址 → 中断处理 → 进程调度 → 持久化文件系统

这正是真实操作系统内核的核心骨架。